

Comunicación Bloqueante y no Bloqueante en MPI

En MPI (Message Passing Interface), las operaciones de comunicación punto a punto se clasifican en bloqueantes y no bloqueantes. Esta distinción es fundamental para el rendimiento de aplicaciones paralelas, ya que determina cómo los procesos esperan (o no) a que las operaciones de comunicación se completen.

Comunicación Bloqueante

Una rutina bloqueante no retorna el control al programa hasta que la operación de comunicación se ha completado de forma segura.

En el envío (MPI_Send): La función retorna cuando el buffer de aplicación puede reutilizarse de manera segura. Esto no significa necesariamente que el mensaje haya sido recibido, solo que el sistema ya no necesita el buffer del emisor.

En la recepción (MPI_Recv): La función retorna solo cuando los datos han llegado y están listos para ser usados por el programa

Ejemplo: Atención en una Tienda de Café

Escenario: Una cafetería con un solo barista (Proceso A) y varios clientes (Procesos B,C,D)

El barista atiende un solo cliente a la vez. Mientras prepara el café, no puedes hacer otra cosa:

Cliente 1: "Quiero un café"

— Barista: Prepara café (no puede atender al Cliente 2)

— Barista: "Toma tu café" (bloqueo termina)

Cliente 2: "Quiero un té" → Barista: Prepara té...

Equivalente en MPI:

```
// Barista (proceso 0)
while(1) {
    MPI_Recv(&pedido, 1, MPI_INT, MPI_ANY_SOURCE, 0,
MPI_COMM_WORLD, ...);
    // Bloqueado hasta que llega un pedido
    preparar_bebida(pedido); // Solo esto puede hacer
    MPI_Send(&bebida, 1, MPI_INT, cliente, 0, MPI_COMM_WORLD);
    // Bloqueado hasta enviar
}
```

Comunicación no Bloqueante

Las rutinas no bloqueantes (MPI_Isend, MPI_Irecv) retornan inmediatamente, sin esperar a que la comunicación se complete. Proporcionan un “manejador” (MPI_Request) que permite verificar el estado posteriormente.

Características:

- No es seguro modificar el buffer hasta que se complete la operación (usar MPI_Wait o MPI_Test)
- Permiten solapar computación con comunicación.
- Requieren código adicional para gestionar las peticiones pendientes.

Ejemplo: Cocina de Restaurante

Escenario: Un chef (Proceso 0) prepara platos, un ayudante (Proceso 1) trae ingredientes.

El chef inicia la solicitud de ingredientes y mientras llega, sigue cocinando:

Chef: "Ayudante, necesito 5kg de tomates" (inicia solicitud, no espera)

Chef: Sigue picando cebollas, calentando sartenes (trabajo útil)

[El ayudante trae los tomates mientras el chef trabaja]

Chef: "Ah, ya llegaron los tomates" (completa la espera)

Chef: Continúa con la salsa

```
// Chef (proceso 0)
MPI_Isend(&pedido, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, &request);
// Regresa rapido
cortar_cebollas(); // Trabajo mientras el ayudante trae tomates
calentar_sarten();
MPI_Wait(&request, MPI_STATUS_IGNORE); // Ahora sí espera si es necesario
hacer_salsa();

// Ayudante (proceso 1)
MPI_Irecv(&pedido, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &request);
ir_al_almacen(); // Trabajo mientras espera la solicitud
MPI_Wait(&request, ...); // Ahora sí recibe qué traer
traer_ingredientes(pedido);
```

Usar comunicación bloqueante cuando:

- El programa es simple y el rendimiento no es crítico.
- No hay oportunidad de solapar computación (por ej. comunicaciones secuenciales)
- El tamaño de mensaje es pequeño

Usar comunicación no bloqueante cuando:

- Se necesita máximo rendimiento
- Hay computación útil que realizar mientras se comunica
- Se quiere evitar deadlocks en intercambios complejos

